

Shahjalal University Of Science and Technology, Sylhet

THESIS DEFENSE

Presented By
Towhidur Rahman Asif
&
M. M. Zahid Hasan

The Role of Structure in Reasoning: A Comparative Evaluation of Structured Chain-of-Thought Prompting for Text-to-SQL with Modern Language Models

Towhidur Rahman Asif (2020831016)
&
M. M. Zahid Hasan (2020831035)

Supervisor:
Fazle Rabbi Rakib

Lecturer
Department of Software Engineering
Shahjalal University of Science and Technology, Sylhet, Bangladesh

Date: 25 November, 2025

Abstract

Large Language Models have improved Text-to-SQL tasks, but they still struggle with complex, multi-step reasoning. To address this, we propose Structured Chain-of-Thought (SCoT), which guides the model through a clear, structured process. We compare SCoT with several prompting methods and find that it performs the best across all settings. On Gemini 2.5 Pro and GPT-4o-mini, SCoT reaches 84.33% and 70.20% execution accuracy, clearly outperforming baselines. Our seven-stage pipeline produces no execution errors, showing that light structure and multiple candidate solutions can make Text-to-SQL far more reliable without extra training or annotation.

Overview

1 Introduction

2 Research Questions

3 Literature Review

4 Methodology

5 Results

6 Comparison

7 Findings

8 Conclusion



Introduction

Motivation & Background

- SQL is hard for non-technical users.
- NLIs allow users to query databases using natural language.
- Text-to-SQL converts natural language questions into SQL.

Shift Toward Large Language Models

- Traditional Text-to-SQL relies on supervised models requiring costly, large annotated datasets.
- LLMs reduce this dependency—they can perform tasks through prompting, not task-specific training.
- This makes development faster, cheaper, and more flexible.

Core Challenge: Complex Reasoning

- Many Text-to-SQL queries require multi-step logical reasoning.
- Chain-of-Thought (CoT) prompting helps break problems into steps, improving performance.
- But CoT is unstructured, and for highly complex queries it may drift logically, leading to incorrect SQL.
- In benchmarks like Spider, this limitation becomes evident.

Research Questions

1

RQ1:

Can Structured Chain-of-Thought prompting directly improve large language models' accuracy and reasoning in generating SQL queries?

2

RQ2:

Does enforcing a structured reasoning format (SCoT) lead to higher exact match accuracy in Text-to-SQL generation compared to unstructured Chain-of-Thought (CoT)?

3

RQ3:

Howdoestheperformance of SCoT compare against other advanced prompting techniques, including Question Decomposition (QDecomp), across a variety of modern Large Language Models?

Literature Review

This chapter reviews prior work on Text-to-SQL, large language models, and prompting-based reasoning methods, and positions our proposed SCoT approach within this landscape.

1

Literary review 1

- Early semantic parsing on small, single-domain datasets (e.g., GEOQUERY).
- Spider benchmark introduced cross-domain, multi-table, complex SQL queries.
- LLMs with in-context learning enable few/zero-shot Text-to-SQL.

2

Literary review 2

- CoT helps reasoning but can propagate early SQL clause errors.
- QDecomp and QDecomp+InterCOL use higher-level sub-questions with schema grounding.
- They outperform standard prompting and CoT on Spider benchmarks.
- Reasoning steps are still unstructured text → ambiguity remains.
- This motivates our Structured Chain-of-Thought (SCoT) method.

Methodology

1. **Datasets:** Spider benchmark subsets (dev_600 for Gemini 1.5 Pro, dev_500 for GPT-4o-mini); human-readable schema format with PK/FK.
2. **Baselines:** Standard (direct SQL), CoT (step-by-step reasoning), QDecomp (question decomposition), QDecomp_InterCol (with PK/FK grounding).
3. **SCoT Stages:** Plan generation in JSON, validation (schema/PK-FK paths), pattern-aware candidates (4-7 diverse SQL), sanitization & auto-fixes.
4. **SCoT Stages (cont.):** GROUP BY/HAVING validators, template skeletons, canonicalization, execution-guided ranking with cardinality checks.
5. **Protocol:** SQLite execution environment; hyperparameters (τ=0.0-0.2, max tokens 1024); metrics: Execution Accuracy (EX) for semantic match.

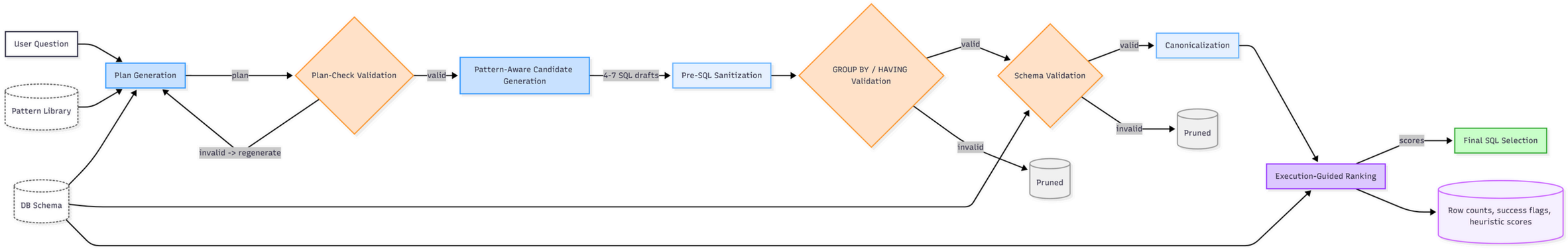
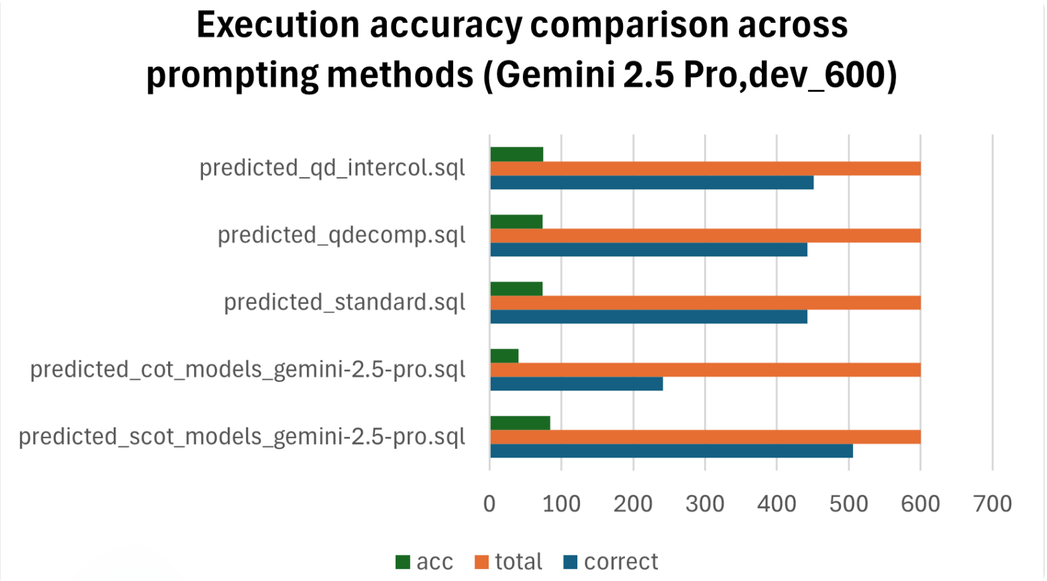


Figure :- SCoT pipeline flowchart (Question/Schema → Plan → Validation → Generation → Ranking → Final SQL).

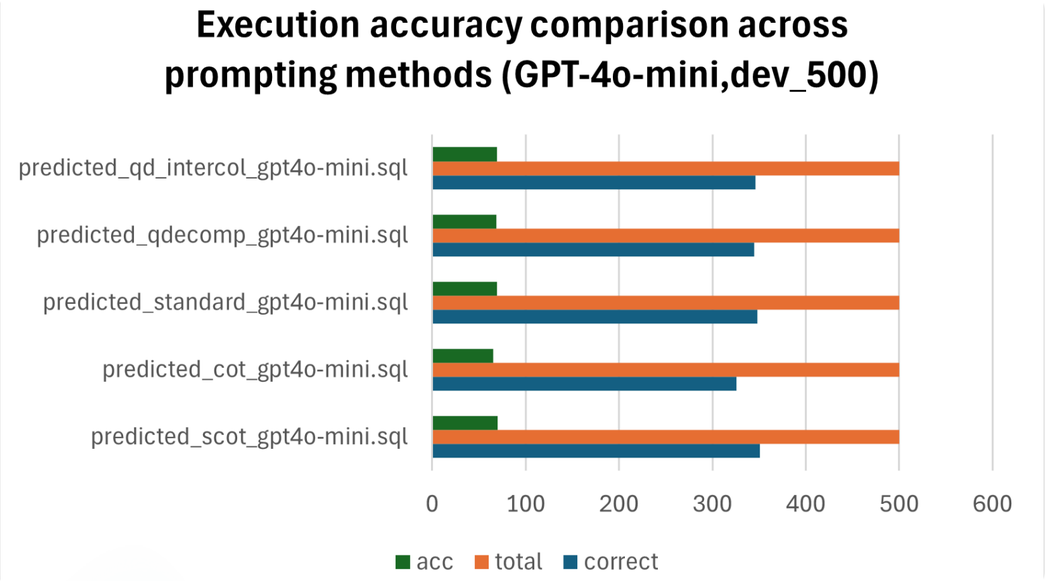
Results



Gemini 2.5 Pro Execution Accuracy

- SCoT achieves 84.33% EX, the highest among all methods.
- ~9% higher than the strongest baseline (75.17%).
- Standard and QDecomp perform similarly (73.67% EX).
- Chain-of-Thought is the weakest (40.17% EX), confirming its instability on complex queries.

Results



GPT-4o-mini Execution Accuracy

- SCoT again ranks first with 70.20% EX.
- Other methods cluster closely:
  Standard: 69.60%,
  QDecomp_InterCol: 69.20%,
  QDecomp: 69.00%
- Chain-of-Thought is still lowest (65.20% EX).

Results

Cross-Model Insights

- SCoT is the top performer on both models, with consistent gains.
- The improvement gap is large on Gemini (~9 points) but small on GPT-4o-mini (~1 point).
- Chain-of-Thought remains the least accurate, supporting evidence that long, clause-level reasoning introduces errors.
- On GPT-4o-mini, Standard and QDecomp variants cluster tightly, showing that the lighter model reduces method differences.

Comparision of SCoT vs CoT

1) SCoT uses a structured JSON format, whereas CoT relies on free-text reasoning.

2) Multiple candidate reasoning paths (4–7) allow SCoT to explore diverse logical routes; CoT generates only a single chain.

3) Built-in schema and logic validation in SCoT prevents invalid reasoning early; CoT has no validation.

4) SCoT includes automatic repair, reducing execution errors; CoT requires manual correction.

Aspect	CoT	SCoT
Format	Free Text	JSON
Candidates	Single	Multiple (4–7)
Validation	None	Schema / Logic
Repair	Manual	Auto-Fix
Accuracy (Gemini)	40.17%	**84.33%**
Accuracy (openAI)	65.22%	**70.19%**

5) Accuracy gains are significant — SCoT nearly doubles accuracy on Gemini (84.33% vs 40.17%) and improves results on GPT-4o-mini.

Findings

- Structured reasoning made a much larger difference than expected.
- Traditional CoT and QDecomp variants often failed due to missing schema grounding and lack of validation.
- SCoT outperformed all baselines because it uses:

  1) JSON-structured reasoning,
  2) multiple candidate paths, and
  3) schema/logic validation with auto- repair.

- These features reduced error propagation and improved accuracy across both models, with the largest gains seen on Gemini.

Aspect	Standard	SCoT	QDecomp	QDecomp+InterCol
Format	Free Text	JSON	NL Steps	NL Steps + Schema Hints
Candidates	Single	Multiple (4-7)	Single	Single
Validation	None	Schema / Logic	None	Light Schema
Repair	None	Auto-Fix	None	None
Accuracy (Gemini)	73.67%	84.33%	73.67%	75.17%
Accuracy (OpenAI)	69.60%	70.19%	69.00%	69.20%

Conclusion

SCoT-based prompting consistently outperformed all other methods on both Gemini 2.5 Pro and GPT-4o-mini. Its structured reasoning, multiple candidate generation, and auto-fix mechanism helped avoid the clause-level errors that affected other approaches. Although Standard and QDecomp variants remained competitive, traditional Chain-of-Thought performed the weakest due to its overly detailed reasoning steps. Overall, our results show that adding structure to the reasoning process makes Text-to-SQL generation more accurate and reliable across different model sizes.

THANK YOU

...